

199. Binary Tree Right Side View

Problem Summary

Given the root of a binary tree, return the values of nodes visible when looking at the tree from the right side — the rightmost node of each level.

Approach

BFS level order traversal — for each level, only add the last node ($i == \text{levelSize} - 1$) to the result.

Algo

1. If $\text{root} == \text{null}$ return empty list.
2. Add root to queue.
3. While queue not empty:
 - Snapshot $\text{levelSize} = \text{q.size}()$.
 - For each node in level: poll, add last node's value to result, add children.
4. Return result.

Key Insights

- $i == \text{levelSize} - 1$ identifies the last (rightmost) node of each level.
- levelSize snapshot before inner loop is essential — queue grows as children are added.
- Left side view = $i == 0$ — single character change.
- BFS naturally processes left to right — last polled node per level is rightmost.

Time Complexity & Space Complexity

$O(n)$ — every node visited once.

$O(n)$ — queue holds at most one full level.

Observations

- DFS alternative: traverse right subtree before left, track depth, add first node seen at each depth.
- No extra list per level needed — just track the last node per level.
- Works correctly for unbalanced trees — BFS always captures the actual rightmost visible node.

Problem List

199. Binary Tree Right Side View

Solved

Medium

Topics

Companies

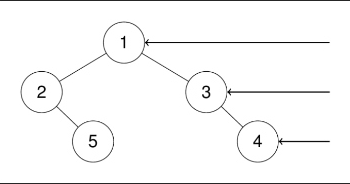
Given the `root` of a binary tree, imagine yourself standing on the **right side** of it, return the values of the nodes you can see ordered from **top to bottom**.

Example 1:

Input: `root = [1,2,3,null,5,null,4]`

Output: `[1,3,4]`

Explanation:

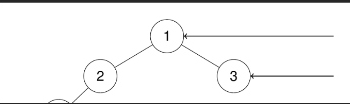


Example 2:

Input: `root = [1,2,3,4,null,null,null,5]`

Output: `[1,3,4,5]`

Explanation:



Accepted

217 / 217 testcases passed

Sahil Khan submitted at Jul 31, 2026 22:03

Unlock the Full LeetCode Experience

Runtime: 1 ms | Beats 70.40%

Memory: 43.48 MB | Beats 85.38%

Code | Java

```
1 /**
2  * Definition for a binary tree node.
3  * public class TreeNode {
4  *     int val;
5  *     TreeNode left;
6  *     TreeNode right;
7  *     TreeNode() {}
8  *     TreeNode(int val) { this.val = val; }
9  * }
```

Code

Auto

```
14 * }
15 */
16 class Solution {
17     public List<Integer> rightSideView(TreeNode root) {
18         List<Integer> result = new ArrayList<>();
19         if(root == null) return result;
20
21         Queue<TreeNode> q = new LinkedList<>();
22         q.offer(root);
23
24         while(!q.isEmpty()){
25             int levelSize = q.size();
26             for(int i=0; i<levelSize; i++){
27                 TreeNode curr = q.poll();
28
29                 if(i==levelSize - 1) result.add(curr.val);
30
31                 if(curr.left != null) q.offer(curr.left);
32                 if(curr.right != null) q.offer(curr.right);
33             }
34         }
35
36         return result;
37     }
38 }
39 }
```

Testcase

Test Result

Accepted

Runtime: 0 ms

Case 1 Case 2 Case 3 Case 4

Input

root = [1,2,3,null,5,null,4]

Output